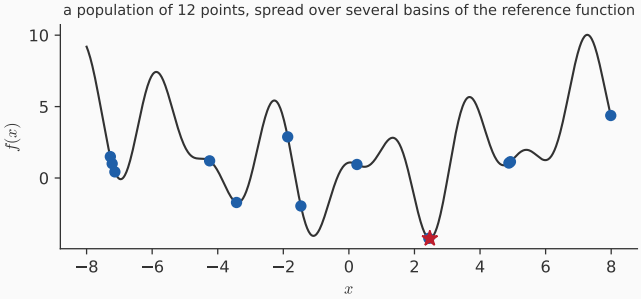
Optimization

5.3 Genetic algorithms

José Firmino Aguilar Madeira

From SA to GA: what if we have a population?

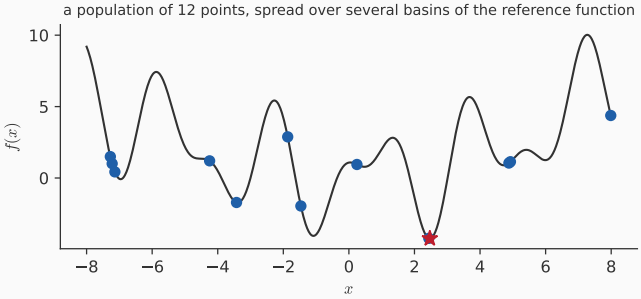


GLODS (5.1): several points + local searches.

SA (5.2): one point that can move uphill.

GA (5.3): a population that evolves and exchanges information.

From SA to GA: what if we have a population?



GLODS (5.1): several points + local searches.
SA (5.2): one point that can move uphill.
GA (5.3): a population that evolves and exchanges information.

Question: how do we make the population progressively concentrate on the better regions without losing diversity too early?

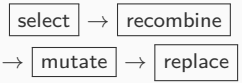
This question contains practically the whole GA.

Idea and vocabulary

Holland (1975), Goldberg (1989). The terminology comes from genetics; computationally, we have solutions, variables, selection, recombination and perturbation.

GA language	In optimization
individual, chromosome	a solution $\mathbf{x}$ (encoded)
gene, allele	variable (or bit) and its value
fitness	$F(\mathbf{x})$: larger = better
generation	iteration
selection	pick parents (prob. grows with F)
crossover	recombine two parents
mutation	perturb an offspring

- The analogy is language, not the mathematical reason why the method works.
- Uses only values of f (order 0); continuous, integer, binary, permutations.
- Like SA: a stochastic heuristic, with no guarantee of a global optimum in finite time.



Encoding and fitness

Binary (classical GA)

$x \in [x^{(L)}, x^{(U)}]$ with ℓ bits: the string $b_{\ell-1} \dots b_0$ represents

$$x = x^{(L)} + \left(x^{(U)} - x^{(L)}\right) \frac{\sum_i b_i 2^i}{2^\ell - 1},$$

with resolution $(x^{(U)} - x^{(L)})/(2^\ell - 1)$. For accuracy ε_x: $\ell \geq \log_2\left(1 + (x^{(U)} - x^{(L)})/\varepsilon_x\right)$ (ceiling). E.g.: $[-5, 5]$, $\varepsilon_x = 10^{-4}$: $\ell = 17$. n variables: concatenate the n strings.

Real

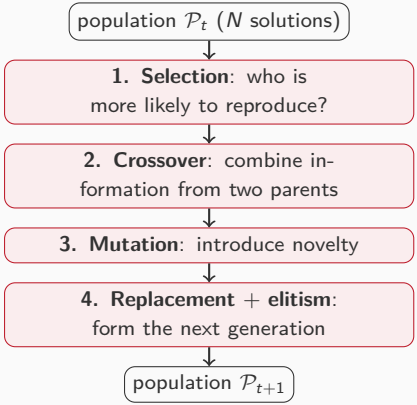
The chromosome is the vector $\mathbf{x} \in \mathbb{R}^n$ itself; the operators work with real numbers. It is the usual choice today in continuous optimization (MATLAB's ga).

Fitness: maximize, with values ≥ 0

For $\min f$: $F = 1/(1+f)$ if $f \geq 0$; or $F = f_{\max} - f$ (in the population); or by ranking (F = position in the sorted list, with the best receiving the largest F) — immune to the scale of f and to extreme values.

- Constraints: penalty (4.3.1) in the fitness, or repair/reject the infeasible ones (4.3.3).
- Binary encoding gives the most transparent example (coming next); its difficulties (fixed precision, "Hamming cliffs": 0111 → 1000 changes every bit) motivate the real one.

One generation in four steps



The tension of the whole GA

Selection focuses the search on what already looks good; crossover and mutation generate new solutions, and mutation is an important source of diversity.

exploitation $\longleftrightarrow$ exploration

The same tension as in the chapter opening — now managed by operators on a population.

Selection: who reproduces?

Roulette wheel

Parent i with probability

$$p_i = \frac{F_i}{\sum_j F_j}.$$

Scale-sensitive: an individual much better than the others takes almost the whole wheel.

Tournament

Draw s individuals (typically 2) and keep the best. Uses only comparisons; the pressure is controlled by s.

Elitism

Copy the e best directly into the next generation: the best value found is not lost.

selection pressure too strong $\rightarrow$ loss of diversity $\rightarrow$ premature convergence

More important than memorizing variants.

Crossover and mutation: how to create novelty?

Crossover (with probability $p_c \approx 0.6$–0.9)

Binary, 1-point: cut both strings at the same position and swap the tails,
100|01000, 000|10111 → 100|10111, 000|01000.

Real, BLX-β: $\mathbf{x} = a\,\mathbf{x}_a + (1-a)\,\mathbf{x}_b$, with $a \sim U(-\beta, 1+\beta)$ per coordinate — it may leave the segment between the parents.

Others: 2-point, uniform, arithmetic, SBX.

Mutation (with probability p_m per gene)

Binary: flip the bit; $p_m \approx 1/(n\ell)$ ($n\ell$ = chromosome length).

Real: add $N(0, \sigma^2)$ to a coordinate.

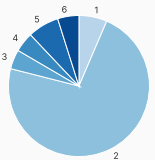
Permutations: swap two positions.

Mutation brings into the population values that did not yet exist in it; crossover recombines those that already exist.

One generation by hand: min x^2 on $[-5, 5]$, 8 bits, $N = 6$, roulette, $p_c = 0.8$, $p_m = 0.05$

Fitness $F = 1/(1 + x^2)$. Random initial population:

i	chromosome	x	$f = x^2$	F	p_i	cumulative
1	11010110	3.392	11.507	0.080	0.064	0.064
2	10001000	0.333	0.111	0.900	0.725	0.790
3	00010110	−4.137	17.117	0.055	0.044	0.834
4	00010111	−4.098	16.794	0.056	0.045	0.879
5	00101110	−3.196	10.215	0.089	0.072	0.951
6	11100100	3.941	15.533	0.060	0.049	1.000



The roulette wheel: slices proportional to F_i.

One generation by hand: min x^2 on $[-5, 5]$, 8 bits, $N = 6$, roulette, $p_c = 0.8$, $p_m = 0.05$

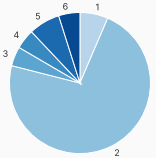
Fitness $F = 1/(1 + x^2)$. Random initial population:

i	chromosome	x	$f = x^2$	F	p_i	cumulative
1	11010110	3.392	11.507	0.080	0.064	0.064
2	10001000	0.333	0.111	0.900	0.725	0.790
3	00010110	−4.137	17.117	0.055	0.044	0.834
4	00010111	−4.098	16.794	0.056	0.045	0.879
5	00101110	−3.196	10.215	0.089	0.072	0.951
6	11100100	3.941	15.533	0.060	0.049	1.000

Roulette: 6 numbers $\xi \sim U(0, 1)$ and the parent is the first i with cumulative $\geq \xi$:

ξ	0.679	0.870	0.227	0.895	0.872	0.019
parent	2	4	2	5	4	1

Individual 2 ($p = 0.73$) is drawn twice; 3 and 6 are not chosen.



The roulette wheel: slices proportional to F_i.

- With $F = 1/(1 + f)$ individual 2 takes almost the whole wheel: roulette selection loses diversity quickly. Tournament or ranking avoid this.

The same generation: crossover and mutation — and the best got worse

1-point crossover, pairs (2,4), (2,5), (4,1); for each pair we draw $\xi < p_c = 0.8$ and the cut position:

parents	chromosomes	cut	offspring
2, 4	10001000, 00010111	cut after bit 3	10010111, 00001000
2, 5	10001000, 00101110	cut after bit 4	10001110, 00101000
4, 1	00010111, 11010110	cut after bit 7	00010110, 11010111

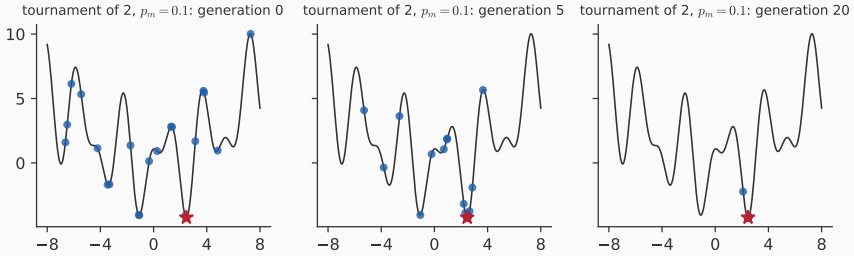
Mutation with $p_m = 0.05$ per bit (48 bits, 2.4 flips expected; one occurred, in red):

i	before	after	x	$f = x^2$
1	10010111	10010111	0.922	0.849
2	00001000	00001100	−4.529	20.516
3	10001110	10001110	0.569	0.323
4	00101000	00101000	−3.431	11.774
5	00010110	00010110	−4.137	17.117
6	11010111	11010111	3.431	11.774

Mean of f: $11.9 \to 10.4$ — but the best got worse ($0.111 \to 0.323$)

Individual 2 was "undone" by crossover. This is exactly what elitism is for: copying the best into the next generation. With elitism and mutation that reaches all of X: asymptotic guarantee, in probability (Rudolph, 1994); none in finite time.

Why a population? Diversity



The reference function, $N = 20$, tournament of 2, BLX-0.5, $p_m = 0.1$: generations 0, 5 and 20.

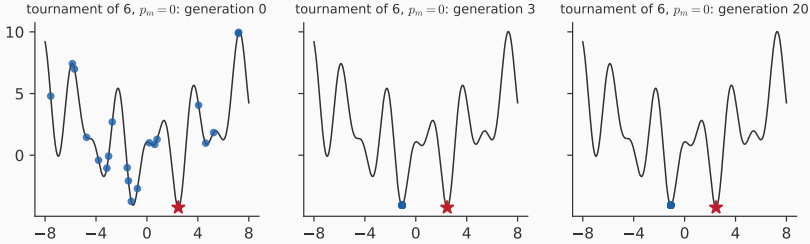
SA: a single trajectory has to discover the basins one by one.

GA: different individuals occupy different regions at the same time.

But selection makes the population concentrate: by generation 20 they are all in the global basin. It went well — because diversity lasted long enough.

The fundamental resource of a GA is the diversity of the population.

The danger: everyone alike too early



The same function, tournament of 6 (strong pressure) and no mutation: within 3 generations the population closed in on the basin of $x \approx -1.08$ (local minimum).

Premature convergence

Spread-out initial population → strong selection → all in the same basin. Without diversity, there is no more global exploration.

Mechanisms that preserve it:

- larger population;
- mutation;
- moderate selection pressure (small tournament, ranking instead of roulette);
- niching, fitness sharing — just to know they exist.

Algorithm

GA: N, p_c, p_m, G generations

1: generate an initial population $\mathcal{P}_0$ of N random individuals; evaluate f (and F)
2: **for** $t = 0, \ldots, G-1$ **do**
3: **selection**: choose N parents from $\mathcal{P}_t$ (roulette/-tournament)
4: **crossover**: in pairs, with prob. p_c, generate two offspring
5: **mutation**: each gene of each offspring, with prob. p_m; project onto X
6: evaluate the offspring
7: **replacement**: set $\mathcal{P}_{t+1}$ to the N offspring, with e of them replaced by the e best of $\mathcal{P}_t$ (elitism)
8: **end for**
9: **return** the best of $\mathcal{P}_G$

- Cost: $n_f = N$ per generation; $n_f = N(G+1)$ in total. $N \approx 20$–200.
- Stopping: fixed G; or stagnation of the best for k generations; or population diversity below a threshold.
- Parameters to tune: N, p_c, p_m, σ, s, e. There are no universal values; small p_m and high p_c are the usual starting point.

The N evaluations of a generation are independent

$\rightarrow$ parallelization: with N processors and evaluations of similar cost, the time of a generation can approach the time of one evaluation. A practical difference from SA.

What do the parameters do?

parameter	small	large
N	little diversity	more evaluations per generation
p_c	little recombination	a lot of mixing
p_m	loses novelty	almost random search
pressure (s)	slow evolution	premature convergence
G (budget)	insufficient budget	higher cost

There are no universal values. Usual starting point: high p_c, small p_m, tournament of 2, elitism.

Alongside, N and budget change together; with $n_f = 420$ fixed: $N = 60$ ($G = 6$) 96%, $N = 6$ ($G = 69$) 46%.

On the reference function, 300 runs, success $|x - x^*| < 0.3$:

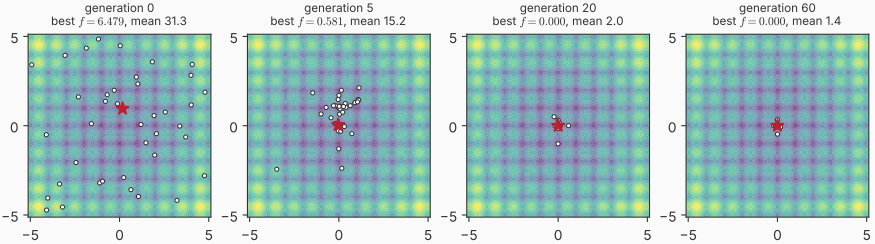
	success	n_f
reference: $N = 20$, $s = 2$, $p_m = 0.1$	75%	420
strong pressure: $s = 6$	61%	420
no crossover: $p_c = 0$	60%	420
small population: $N = 6$	34%	126
larger population: $N = 60$	96%	1260

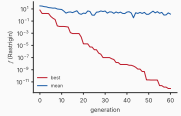
In this example N matters most: a larger population increases the initial diversity. p_m hardly changes the result ($p_m = 0$: 76%; 0.8: 78%) — with BLX-0.5, crossover already generates enough novelty *on this problem*.

(The same configuration gave 71% in 100 runs in the chapter table: the success rate is also an estimate.)

Rastrigin: exploring or already converging?

$f(\mathbf{x}) = 10n + \sum_i \left(x_i^2 - 10\cos 2\pi x_i\right)$, $n = 2$: 121 local minima on a grid, global at $\mathbf{0}$. $N = 40$, tournament of 2, BLX-0.5 ($p_c = 0.9$), Gaussian mutation ($p_m = 0.1$, $\sigma = 0.5$), 2 elites, 60 generations (2440 eval.).

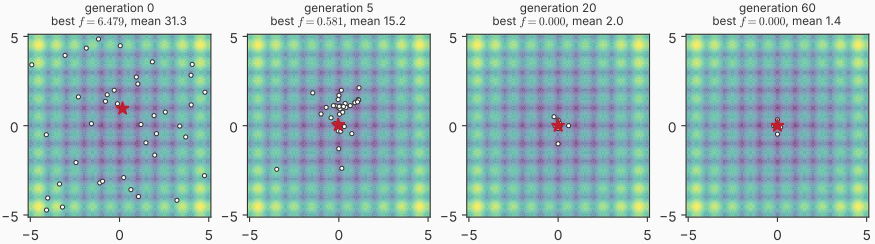


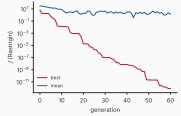


Question: looking at generations 0 and 5 — are we exploring or already converging?

Rastrigin: exploring or already converging?

$f(\mathbf{x}) = 10n + \sum_i \left(x_i^2 - 10\cos 2\pi x_i\right)$, $n = 2$: 121 local minima on a grid, global at $\mathbf{0}$. $N = 40$, tournament of 2, BLX-0.5 ($p_c = 0.9$), Gaussian mutation ($p_m = 0.1$, $\sigma = 0.5$), 2 elites, 60 generations (2440 eval.).





The spread-out population concentrates in the global basin around generation 20 and then refines: $f < 10^{-12}$. The mean (blue) stays above the best: mutation maintains some spread.

The price of genetic algorithms

Advantages

- Explores several regions simultaneously.
- Flexible representation: continuous, discrete, binary, permutations.
- The N evaluations of a generation are independent: parallelizable.
- Natural extension to several objectives.

Limitations

- Many evaluations; many parameters.
- Premature convergence.
- Stochastic result.
- High accuracy is expensive.
- No certificate of globality with a finite budget.
- Does not require derivatives; with noisy evaluations, additional mechanisms are needed to compare individuals reliably.

GA $\rightarrow$ local method

On the reference function: 5 generations ($n_f = 120$) bring the best to $|x - x^*| \approx 3 \times 10^{-3}$; a direct search takes it to 2×10^{-7} with 39 evaluations. Total 159, against 420 for 20 generations (one run; in 300 runs: 70% vs. 75%). The GA finds the good region; the local method refines.

GLODS, SA and GA: what changed?

	GLODS (5.1)	SA (5.2)	GA (5.3)
State	several points	one point	a population
How it escapes	*search* elsewhere	accepts uphill moves	selection + recombination + mutation
Local search	direct search	not explicit (random walk)	none, unless hybridized
Memory	list of minima	current (+ best)	the population (and elites)
Random?	depends on the *search*	yes	yes
Natural output	several minima	the final point (or the best)	the population's best

Three answers to the same question: how to escape a local basin — and at what cost.

On the computer: MATLAB implementation (real encoding)

The core of `GeneticAlgorithm(f, n, lb, ub, N, G, pc, pm, sigma, e)`:

```
for t = 1:G
  [~, o] = sort(F);  E = P(o(1:e), :);      % elites
  i1 = randi(N,N,1);  i2 = randi(N,N,1);    % tournament
  pais = P(i1,:);  m = F(i2) < F(i1);
  pais(m,:) = P(i2(m),:);
  Q = cruzarBLX(pais, pc);                  % BLX-0.5
  M = rand(N,n) < pm;                       % mutation
  Q = Q + M.*sigma.*randn(N,n);             % Gaussian
  Q = min(max(Q,lb),ub);  Q(1:e,:) = E;     % X; elites
  P = Q;  F = arrayfun(@(i) f(P(i,:)), 1:N)';
end
```

The full function (`GeneticAlgorithm`, with the BLX crossover, the initial population and `info.nfev`) is in the course companion code.

Call (Rastrigin, 30 runs):

```
f = @(x) 20 + sum(x.^2 - 10*cos(2*pi*x));
for s = 1:30
  rng(s);
  [x, fx, info] = GeneticAlgorithm(f, ...
     2, -5.12, 5.12, 40, 60, 0.9, 0.1, 0.5, 2);
  fs(s) = fx;              % info.nfev = 2440
end
median(fs), prctile(fs, [25 75]), mean(fs < 1)
```

MATLAB has ga (Global Optimization Toolbox): `ga(f, n, [], [], [], [], lb, ub)`; it accepts constraints and integer variables.

THE SAME IN PYTHON

`scipy.optimize.differential_evolution(f, bounds)` (evolutionary, with a population) and `dual_annealing(f, bounds)`; `r.nfev` counts the evaluations. There is no classical GA in SciPy.

What if there is no single way to rank “better”?

One objective: $f(\mathbf{x}_a) < f(\mathbf{x}_b)$ is enough to select.

What if there is no single way to rank “better”?

One objective: $f(\mathbf{x}_a) < f(\mathbf{x}_b)$ is enough to select.

Several objectives:
$\mathbf{f}(\mathbf{x}) = (f_1(\mathbf{x}), \ldots, f_m(\mathbf{x}))$. It may happen that, between $\mathbf{x}_a$ and $\mathbf{x}_b$, neither is better in all objectives: $\mathbf{x}_a$ is better in f_1, $\mathbf{x}_b$ in f_2. How do we select?

What if there is no single way to rank "better"?

One objective: $f(\mathbf{x}_a) < f(\mathbf{x}_b)$ is enough to select.

Several objectives:
$\mathbf{f}(\mathbf{x}) = (f_1(\mathbf{x}), \ldots, f_m(\mathbf{x}))$. It may happen that, between $\mathbf{x}_a$ and $\mathbf{x}_b$, neither is better in all objectives: $\mathbf{x}_a$ is better in f_1, $\mathbf{x}_b$ in f_2. How do we select?

Answer (Ch. 6)

dominance + diversity — NSGA-II (6.6) is a GA in which selection uses the dominance rank and a measure of spacing (crowding).

The population, which here serves to explore several basins, then serves to approximate a whole Pareto front.

The same function, several strategies

The reference function (1D, $[-8, 8]$, chapter opening), 100 runs with random starting points, success $|x - x^*| < 0.3$:

	success	n_f
Local method (1 start)	14%	36 [34, 38]
Multistart (8 starts)	72%	290 [280, 299]
Teaching GLODS (deterministic)	yes	326
SA ($T_0 = 3$, $\alpha = 0.8$, $n = 10$)	33%	750 [671, 849]
GA ($N = 20$, $G = 20$)	71%	420

(n_f: median [Q1, Q3]; GLODS: one run, which returns the list of 5 interior minima; SA: final point, random starting points — in 5.2, always $x_0 = 6$: 60%.)

Rastrigin 2D (30 runs, success $f < 1$): Nelder–Mead $n_f = 90$ [85, 98] (median [Q1, Q3]), 3%; SA 5323 [4196, 6070], 97%; GA 2440, 100%.

On this problem, with these parameters — this is not a general ranking: the numbers change with the budget and the parameters.

What we see in both cases: escaping from a basin costs evaluations, and stochastic methods can only be described with several runs.

Summary table — Chapter 5

Method	Global exploration	Cost	Guarantee	Randomness	Main limitation
Multistart	a local method from several points	one local search per start	that of each local search	that of the starting points	starts in the same basin repeat work
GLODS (5.1)	several direct searches + *search*, reducing redundancy by proximity	$\leq 2n$ per *poll* + *search*; budget $N_{\max}$	(Clarke) stationarity of refining limits, if f is Lipschitz and the refining directions dense (with a fixed $D_{\oplus}$: only along those of $D_{\oplus}$); list of candidates, no guarantee of finding all basins	deterministic with a deterministic *search*	small basins absorbed by the radius
SA (5.2)	one point that accepts getting worse	one evaluation per proposal; costly at low T	current: asymptotic (finite space, logarithmic cooling: impractical); best visited: like random search (3.2.1)	yes	T_0, α, n, σ sensitive; final accuracy
GA (5.3)	population, recombination, mutation	$n_f = N(G+1)$; parallelizable	elitist GA: asymptotic, in probability, if mutation reaches all of X (Rudolph, 1994); none in finite time	yes	many parameters; accuracy costs generations

All use only values of f (order 0), except multistart, which inherits the information of the local method.

When each strategy makes sense: smooth f with derivatives → Ch. 3 (with multistart); black box and few minima → direct search / GLODS; multimodal/rugged landscape or discrete variables → SA / GA are natural alternatives.

For study. In class: the next two pages. Rudolph, G., *Convergence analysis of canonical genetic algorithms*, IEEE Trans. Neural Networks 5(1):96–101, 1994.

KEY TAKEAWAYS — 5.3

1. **Population:** the GA explores several regions simultaneously.
2. **Selection** + **crossover** + **mutation:** selection concentrates (exploitation); crossover and mutation create novelty (exploration). Excessive selection $\rightarrow$ loss of diversity $\rightarrow$ premature convergence.
3. **Cost:** N evaluations per generation — but parallelizable.
4. **Stochastic:** several runs; no certificate of global optimality with a finite budget.
5. **Hybridization:** the GA finds the good region; a local method refines.

Next: Chapter 5 in summary — and then, with several objectives, Ch. 6.

KEY TAKEAWAYS — Chapter 5

How do we escape from a local basin — and how much does it cost us?

1. A local method tends to converge to the minimum of $\mathbf{x}_0$'s basin; searching globally is exploring the others: **GLODS**, several points + local searches merged by proximity (keeps the basins found); **SA**, one point that accepts getting worse; **GA**, a population exploring simultaneously.
2. Escaping a local minimum costs evaluations of f: several searches, accepted uphill moves or a population.
3. Stochastic methods return a random variable: report $\geq$ 20–30 runs, fixed seed, median and spread.
4. Without additional structural assumptions, there is no guarantee of identifying the global minimum with a finite budget (GLODS: stationarity, if f is Lipschitz and the refining directions dense; SA: current point only asymptotic (finite space), best visited like random search; elitist GA: asymptotic, in probability, if mutation reaches all of X).
5. A final local refinement (Ch. 3) reduces the budget needed for high accuracy.

Next (Ch. 6): what if there are two things we want to improve at the same time?

References and resources

References Full list: *Course bibliography*, at the end of the course PDF.

- Holland, J. H., *Adaptation in Natural and Artificial Systems*, University of Michigan Press, 1975.
- Goldberg, D. E., *Genetic Algorithms in Search, Optimization, and Machine Learning*, Addison-Wesley, 1989.
- Madeira, J. F. A., *Optimização multi-objectivo da topologia de estruturas com algoritmos genéticos*, PhD thesis in Mechanical Engineering, IST, Universidade Técnica de Lisboa, 2004 — in Portuguese.
- Rudolph, G., *Convergence analysis of canonical genetic algorithms*, IEEE Transactions on Neural Networks 5(1):96–101, 1994.

Resources

- Companion code: `GeneticAlgorithm` (BLX crossover, `info.nfev`). MATLAB: `rng(s)`; `ga` (Global Optimization Toolbox).
- Python: `scipy.optimize.differential_evolution`, `scipy.optimize.dual_annealing` (`r.nfev`).